LLM4Tag: Automatic Tagging System for Information Retrieval via Large Language Models

Ruiming Tang*
tangruiming@huawei.com
Huawei Noah's Ark Lab, China

Chenxu Zhu*
zhuchenxu1@huawei.com
Huawei Noah's Ark Lab, China

Bo Chen*
chenbo116@huawei.com
Huawei Noah's Ark Lab, China

Weipeng Zhang
zhangweipeng12@huawei.com
Huawei Noah's Ark Lab, China

Menghui Zhu
zhumenghui1@huawei.com
Huawei Noah's Ark Lab, China

Xinyi Dai
daixinyi5@huawei.com
Huawei Noah's Ark Lab, China

Huifeng Guo
huifeng.guo@huawei.com
Huawei Noah's Ark Lab, China

arXiv:2502.13481v2 [cs.IR] 6 May 2025

ABSTRACT

Tagging systems play an essential role in various information retrieval applications such as search engines and recommender systems. Recently, Large Language Models (LLMs) have been applied in tagging systems due to their extensive world knowledge, semantic understanding, and reasoning capabilities. Despite achieving remarkable performance, existing methods still have limitations, including difficulties in retrieving relevant candidate tags comprehensively, challenges in adapting to emerging domain-specific knowledge, and the lack of reliable tag confidence quantification. To address these three limitations above, we propose an automatic tagging system LLM4Tag. First, a graph-based tag recall module is designed to effectively and comprehensively construct a small-scale highly relevant candidate tag set. Subsequently, a knowledge-enhanced tag generation module is employed to generate accurate tags with long-term and short-term knowledge injection. Finally, a tag confidence calibration module is introduced to generate reliable tag confidence scores. Extensive experiments over three large-scale industrial datasets show that LLM4Tag significantly outperforms the state-of-the-art baselines and LLM4Tag has been deployed online for content tagging to serve hundreds of millions of users.

CCS CONCEPTS

• **Information Retrieval** → **Tagging Systems**.

KEYWORDS

Tagging Systems; Large Language Models; Information Retrieval

*Co-first authors with equal contributions.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
KDD '2025, August 03–07, 2025, Toronto, ON, Canada
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Tagging is the process of assigning tags, such as keywords or labels, to digital content, products, or users to facilitate organization, retrieval, and analysis. Tags serve as descriptors that summarize key attributes or themes, enabling efficient categorization and searchability, which play a crucial role in information retrieval systems, such as search engines, recommender systems, content management, and social networks [3, 11, 17, 33]. For information retrieval systems, tags are widely used in various stages, including content distribution strategies, ranking algorithms, and operational decision-making processes [2, 7]. Therefore, tagging systems must not only require high accuracy and coverage, but also provide interpretability and strong confidence.

Before the era of Large Language Models (LLMs), the mainstream tagging methods mainly included statistics-based (*i.e.*, TF-IDF-based [24], LDA-based [8]), supervised classification-based (*i.e.*, CNN-based [9, 32], RNN-based [21, 27]), pre-trained model-based methods (*i.e.*, BERT-based [12, 23, 30]), *etc.* However, limited by the model capacity, these methods cannot achieve satisfactory results, especially for complex contents. Besides, they heavily rely on annotated training data, resulting in limited generalization and transferability.

The rise of LLMs, with their extensive world knowledge, powerful semantic understanding, and reasoning capabilities, has significantly enhanced the effectiveness of tagging systems. LLM4TC [5] employs LLMs directly as tagging classifiers and leverages annotated data to fine-tune LLMs. TagGPT [15] further introduces a match-based recall to filter out a small-scale tag set to address the limited input length of LLMs. ICXML [35] proposes an in-context learning algorithm to guide LLMs to further improve performance.

However, existing LLM-enhanced tagging algorithms exhibit several critical limitations that require improvement (shown in Figure 1):

(L1) Constrained by the input length and inference efficiency of LLMs, existing methods adopt simple match-based recall to filter out a small-scale candidate tag set [15, 35], which is prone to missing relevant tags, thereby reducing accuracy.

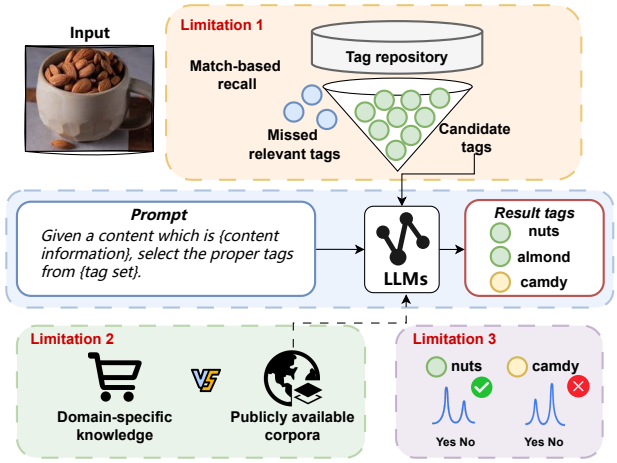


Figure 1: LLM-enhanced tagging systems and their three limitations. (L1) Simple match-based recall is prone to missing relevant tags; (L2) The emerging domain-specific knowledge may not align with the pre-trained knowledge of LLMs; (L3) LLMs cannot accurately quantify tag confidence.

(L2) General purpose LLMs pre-trained in publicly available corpora exhibit limitations in comprehending emerging domain-specific knowledge within information retrieval, leading to lower accuracy in challenging cases [5, 18].

(L3) Due to the hallucination and uncertainty [13, 14], LLMs cannot accurately quantify tag confidence, which is crucial for information retrieval applications.

To address the three limitations of existing approaches, we propose an automatic tagging system called LLM4Tag, which consists of three key modules. Specifically, to improve the completeness of candidate tags (L1), we propose a graph-based tag recall module designed to construct small-scale, highly relevant candidate tags from a massive tag repository efficiently and comprehensively. To enhance domain-specific knowledge and adaptability to emerging information of general-purpose LLMs (L2), a knowledge-enhanced tag generation module that integrates long-term supervised knowledge injection and short-term retrieved knowledge injection is designed to generate accurate tags. Moreover, a tag confidence calibration module is introduced to generates reliable tag confidence scores, ensuring more robust and trustworthy results (L3).

To summarize, the main contributions of this paper can be highlighted as follows:

- We propose an LLM-enhanced tagging framework LLM4Tag, characterized by completeness, continuous knowledge evolution, and quantifiability.
- To address the limitations of existing approaches, LLM4Tag integrates three key modules: graph-based tag recall, knowledge-enhanced tag generation, and tag confidence calibration, ensuring the generation of accurate and reliable tags.
- LLM4Tag achieves state-of-the-art in three large-scale industrial datasets with detailed analysis that provides a deeper understanding of model performance. Moreover, LLM4Tag has been deployed online for content tagging, serving hundreds of millions of users.

2 RELATED WORK

In this section, we briefly review traditional tagging systems and LLM-enhanced tagging systems.

2.1 Traditional Tagging Systems

Traditional tagging systems [6, 11, 22] generally employ multi-label classification models, which utilize human-annotated tags as ground truth labels and employ content descriptions as input to predict. Qaiser et al. [24] utilize TF-IDF to categorize the tag while Diaz et al. [8] employ LDA to automatically tag the resource based on the most likely tags derived from the latent topics identified. The advent of deep learning has led to the proposal of RNN-based [21] and CNN-based [32] methods for achieving multi-label learning, which are directly applied to tagging systems [9, 27]. Hasegawa et al. [12] further adopt the BERT pre-training technique in their tagging systems. Recently, with the growing popularity of pre-trained Small Language Models (SLMs), numerous pre-training embedding models, such as BGE [30], GTE [19], and CONAN [16], have been proposed and directly employed in tagging systems through domain knowledge fine-tuning.

Nonetheless, the capabilities of these models are constrained by their limited model capacity, particularly in the presence of complex content. Additionally, they depend excessively on annotated training data, resulting in sub-optimal generalization and transferability.

2.2 LLM-Enhanced Tagging Systems

With Large Language Models (LLMs) achieving remarkable breakthroughs in natural language processing [1, 4, 10, 26] and information retrieval systems [20, 34], LLM-enhanced tagging systems have received much attention and have been actively explored currently [5, 15, 25, 29, 35]. Wang et al. [29] employ LLMs as a direct tagging classifier, while Sun et al. [25] introduce clue and reasoning prompts to further enhance performance. LLM4TC [5] undertakes studies on diverse LLMs architectures and leverages annotated samples to fine-tune the LLMs. TagGPT [15] introduces an early match-based recall mechanism to generate candidate tags from a large-scale tag repository with textual clues from multimodal data. ICXML [35] proposes a two-stage framework through in-context learning to guide LLMs to align with the tag space.

However, the aforementioned works suffer from three critical limitations (mentioned in Section 1): (1) difficulties in comprehensively retrieving relevant candidate tags, (2) challenges in adapting to emerging domain-specific knowledge, and (3) the lack of reliable tag confidence quantification. To this end, we propose LLM4Tag, an automatic tagging system, to address the aforementioned limitations.

3 METHODOLOGY

In this section, we present our proposed LLM4Tag framework in detail. We start by providing an overview of the proposed framework and then give detailed descriptions of the three modules in LLM4Tag.

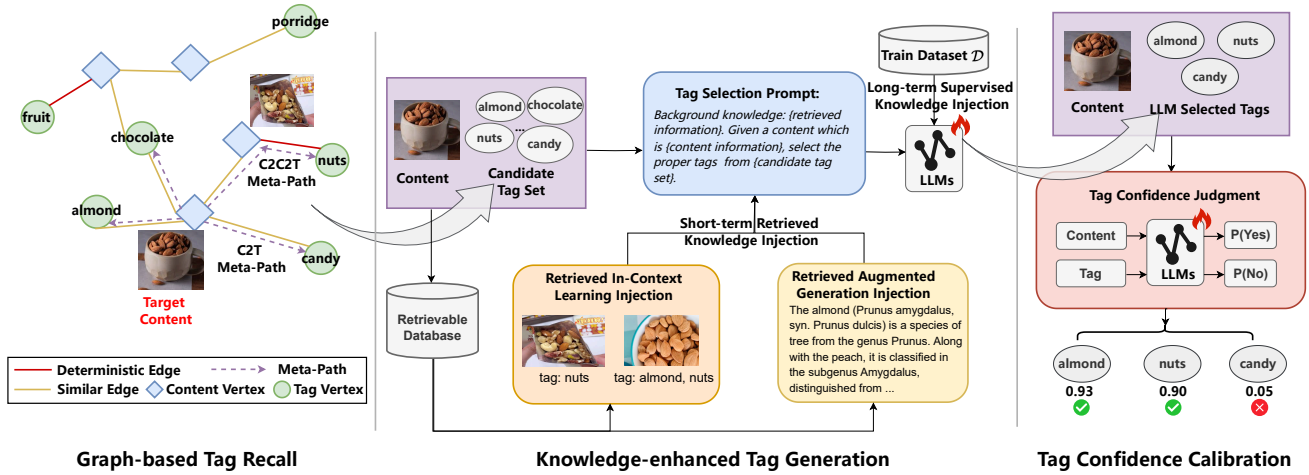


Figure 2: The overall framework of LLM4Tag architecture of LLM4Tag, consisting of three modules: graph-based tag recall module, knowledge-enhanced tag generation module, and tag confidence calibration module.

3.1 Overview

As illustrated in Figure 2, our proposed LLM4Tag framework consists of three major modules: (1) Graph-based Tag Recall, (2) Knowledge-enhanced Tag Generation, and (3) Tag Confidence Calibration, which respectively provides completeness, continual knowledge evolution, and quantifiability.

Graph-based Tag Recall module is responsible for retrieving a small-scale, highly relevant candidate tag set from a massive tag repository. Based on a scalable content-tag graph constructed dynamically, graph-based tag recall is utilized to fetch dozens of relevant tags for each content.

Knowledge-enhanced Tag Generation module is designed to accurately generate tags for each content via Large Language Models (LLMs). To address the lack of domain-specific and emerging knowledge in general-purpose LLMs, this module implements a scheme integrating the injection of both long-term and short-term domain knowledge, thereby achieving continual knowledge evolution.

Tag Confidence Calibration module is aimed to generate a quantifiable and reliable confidence score for each tag, thus alleviating the issues of hallucination and uncertainty in LLMs. Furthermore, the confidence score can be employed as a relevance metric for downstream information retrieval tasks.

3.2 Graph-based Tag Recall

Given the considerable magnitude of tags (millions) in industrial information retrieval system, the direct integration of the whole tag repository into LLMs is impractical due to the constrained nature of the context window and inference efficiency of LLMs. Existing approaches [15, 35] adopt simple match-based tag recall to filter out a small-scale candidate tag set based on small language models (SLMs), such as BGE [30]. However, they are prone to missing relevant tags due to the limited capabilities of SLMs. To address this issue and improve the comprehensiveness of the retrieved candidate tags, we construct a semantic graph globally and propose a graph-based tag recall module.

Firstly, we initial an undirected graph $\mathcal{G}$ with contents and tags as:

$$\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}, \tag{1}$$

where vertex set $\mathcal{V} = \{\mathcal{C}, \mathcal{T}\}$ is the set of existing content vertices $\mathcal{C}$ and all tag vertices $\mathcal{T}$. As for the edge set $\mathcal{E}$, it contains two types of edges, called *Deterministic Edges* and *Similarity Edges*.

Deterministic Edges only connect between content vertex c and tag vertex t, formulated as e^d_{c-t}, which indicates that content c is labeled with tag t based on historical annotation data. To ease the high sparsity of the deterministic edges in the graph $\mathcal{G}$, we further introduce semantic similarity-based edges (**Similarity Edges**) that connect not only between content vertex c and tag vertex t, but also between different content vertices, formulated as e^s_{c-t} and e^s_{c-c}, respectively.

Specifically, for the i-th vertex $v^i \in \mathcal{V}$ in graph $\mathcal{G}$, we summarize all textual information (*i.e.*, title and category in content, tag description) as $text^i$ and vectorize it with an encoder to get a semantic representation $\boldsymbol{r}^i$:

$$\boldsymbol{r}^i = \text{Encoder}(text^i), \tag{2}$$

where Encoder is a small language model, such as BGE [30]. Then the similarity distance of two different vertices v^i, v^j can be computed as:

$$\text{Dis}(v^i, v^j) = \frac{\boldsymbol{r}^i \cdot \boldsymbol{r}^j}{\|\boldsymbol{r}^i\| \|\boldsymbol{r}^j\|}. \tag{3}$$

After obtaining the similarity estimations, we can use a threshold-based method to determine the similarity edge construction, *i.e.*,

- e^s_{c-t} connects the content c and the tag t if the similarity distance between them exceeds δ_{c-t}.
- e^s_{c-c} connects the similar contents when their similar distance exceeds δ_{c-c}.

In this way, we can construct a basic content-tag graph with deterministic/similarity edges. Then, when a new content c appears

that needs to be tagged, we dynamically insert it into this graph by adding similarity edges. Next, we define two types of meta-paths (*i.e.*, C2T meta-path and C2C2T meta-path) and adopt the meta-path-based approach to recall candidate tags.

C2T Meta-Path: Based on the given content c, we first recall the tags which is connected directly to c as the candidate tags. The meta-path can be defined as:

$$p^{C2T} = c \xrightarrow{s} t \,, \tag{4}$$

where $\xrightarrow{s}$ is the similarity edge.

C2C2T Meta-Path: C2C2T contains two sub-procedures: C2C and C2T. C2C is aimed at discovering similar contents while C2T further attempts to recall the deterministic tags from these similar contents. The meta-path can be formulated as:

$$p^{C2C2T} = c \xrightarrow{s} c \xrightarrow{d} t \,, \tag{5}$$

where $\xrightarrow{d}$ is the deterministic edge and $\xrightarrow{s}$ is the similarity edge.

With these two types of meta-paths, we can generate a more comprehensive candidate tag set for content c as

$$\Phi(c) = \Phi^{C2T}(c) \cup \Phi^{C2C2T}(c) \,, \tag{6}$$

where $\Phi^{C2T}(c)$ is retrieved by C2T meta-path and Φ^{C2C2T} is retrieved by C2C2T meta-path. Notably, the final tagging results of LLM4Tag for the content c will also be added to the graph as deterministic edges, enabling dynamic scalability of the graph.

Compared to simple match-based tag recall, our graph-based tag recall leverages semantic similarity to construct a global content-tag graph and incorporates a meta-path-based multi-hop recall mechanism to enhance candidate tags completeness, which will be demonstrated in Sec 4.3.

3.3 Knowledge-enhanced Tag Generation

After obtaining the candidate tag set, we can directly use the Large Language Models (LLMs) to select the most appropriate tags. However, due to the diversity and industry-specific nature of the information retrieval system applications, domain-specific knowledge varies significantly across different scenarios. That is, the same content and tags may have distinct definitions and interpretations depending on the specific application context. Furthermore, the domain-specific knowledge is emerged continually and constantly at an expeditious pace. As a result, the general-purpose LLMs have difficulty in understanding the emerging domain-specific information, such as newly listed products, emerging hot news, or newly added tags, leading to a lower accuracy on challenging cases.

To address the lack of emerging domain-specific information in LLMs, we devise a knowledge-enhanced tag generation scheme that takes into account both long-term and short-term domain-specific knowledge by two key components, namely *Long-term Supervised Knowledge Injection* (LSKI), *Short-term Retrieved Knowledge Injection* (SRKI).

3.3.1 Long-term Supervised Knowledge Injection. For long-term domain-specific knowledge, we first construct a training dataset $\mathcal{D}$ and adopt a basic prompt template as $Template_b$ for tag generation (shown in Figure 3).

Basic Tag Generation Prompt Template

You are an advertising tag bot. Considering an advertisement creative with {image} and {product description}, select the most relevant tag from {candidate tag set}.

Figure 3: Prompt template for basic tag generation in advertisement creatives tagging scenario.

$$\begin{aligned} \mathcal{D} &= \{(x_i, y_i)\}_{i=1}^{N} \,, \\ x_i &= Template_b(c_i, \Phi(c_i)) \,, \end{aligned} \tag{7}$$

where N is the size of training dataset. Notably, to ensure the comprehensiveness of domain-specific knowledge, we employ the principle of diversity for sample selection and obtain correct answers y_i by combining LLMs generation with human expert annotations.

After obtaining the training set, we leverage the causal language modeling objective for LLM Supervised Fine-Tuning (SFT):

$$\max_{\Theta} \sum_{i=1}^{N} \sum_{j=1}^{|y_i|} \log P_\Theta \left(y_{i,j} \mid x_i, y_{i,<j} \right) \,, \tag{8}$$

where Θ is the parameter of LLM, $y_{i,j}$ is the j-th token of the textual output y_i, and $y_{i,<j}$ denotes the tokens before $y_{i,j}$ in the i-th samples.

By adopting this approach, we can effectively integrate the domain-specific knowledge from information retrieval systems into LLMs, thus improving the tagging performance.

3.3.2 Short-term Retrieved Knowledge Injection. Although LSKI effectively provides domain-specific knowledge, continuously incorporating short-term knowledge through LLM fine-tuning is highly resource-intensive, especially given the rapid emergence of new domain knowledge. Additionally, this approach suffers from poor timeliness, making it more challenging to adapt to rapidly evolving content in information retrieval systems, particularly for emerging hot topics.

Therefore, we further introduce a short-term retrieved knowledge injection (SRKI). Specifically, we derive two retrieved knowledge injection methods: *retrieved in-context learning injection* and *retrieved augmented generation injection.*

Retrieved In-Context Learning Injection. We first construct a retrievable sample knowledge base (including contents and their correct/incorrect annotated tags) and continuously append newly emerging samples. Then, given the target content c, this composition retrieves n relevant samples from the sample knowledge base. This approach not only leverages the few-shot in-context learning capability of LLMs but also enables them to quickly adapt to emerging domain knowledge, enhancing tagging accuracy for challenging cases.

Retrieved Augmented Generation Injection. Given the content c and the candidate tag set $\Phi(c)$, this composition retrieves relevant descriptive corpus from web search and domain knowledge base. It can retrieve extensive information that assists LLMs in understanding unknown domain-specific knowledge or new knowledge, such as the definition of terminology in the content/tag or some manually defined tagging rules.

Retrieval Enhanced Tag Generation Prompt Template

You are an advertising tag bot. Considering an advertisement creative with {image} and {product description}. Domain-specific knowledge: (1) Similar advertisement creatives and corresponding tags: {(ad1, tag1), (ad2, tag2), ... }; (2) Information related to the advertisement and tags: {definition of terminology, related tagging rules, ...}. Now you need to select the most relevant tag from {candidate tag set}.

Figure 4: Prompt template for retrieval enhanced tag generation in advertisement creatives tagging scenario.

After obtaining the retrieved knowledge, we design a prompt template, $Template_r$, (shown in Figure 4) to integrate knowledge with the content c and candidate tag set $\Phi(c)$ to provide the in-context guidance for LLMs to predict the most appropriate tags for content c as:

$$\begin{aligned}\Gamma(c) &= \text{LLM}(Template_r(c, \Phi(c), R(c))) \,, \\ &= \left\{t_1^c, t_2^c, \cdots, t_m^c\right\} \,,\end{aligned} \tag{9}$$

where $R(c)$ is the retrieved knowledge above, and m is the number of appropriate tags generated by LLMs.

3.4 Tag Confidence Calibration

After tag generation, there still exist two serious problems for real-world applications: (1) the hallucination due to the uncertainty of LLMs, which leads to generating irrelevant or wrong tags; (2) the necessity of assigning a quantifiable relevance score for each tag for the sake of downstream usage in the information retrieval systems (*e.g.*, recall and marketing).

Tag Confidence Judgment Prompt Template

You are an advertising tag relevance judgment bot. Given an advertisement creative with {image} and {product description} and a tag with {tag description}, judge whether the advertisement is relevant to the tag. Answer with "Yes" or "No".

Figure 5: Prompt template for tag confidence judgment in advertisement creatives tagging scenario.

To handle these two problems, the tag confidence calibration module is adopted. Specifically, given a target content c and a certain tag $t^c \in \Gamma(c)$, we derive a prompt template, $Template_c$ (shown in Figure 5), to leverage the reasoning ability of LLMs to achieve a tag confidence judgment task, *i.e.*, whether c and t^c is relevant. Then we extract the probability of the token in the LLM result to get a confidence score $\text{Conf}(c, t^c)$:

$$\begin{aligned}\boldsymbol{s} &= \text{LLM}(Template_c(c, t^c)) \in \mathbb{R}^V, \\ \text{Conf}(c, t^c) &= \frac{\exp\left(\boldsymbol{s}\left[\text{“}Yes\text{”}\right]\right)}{\exp\left(\boldsymbol{s}\left[\text{“}Yes\text{”}\right]\right) + \exp\left(\boldsymbol{s}\left[\text{“}No\text{”}\right]\right)} \in (0, 1) \,,\end{aligned} \tag{10}$$

where $\boldsymbol{s}$ is the probability score for all tokens, and V is the vocabulary size of LLMs.

After obtaining the confidence score $\text{Conf}(c, t)$, we implement self-calibration for the results by eliminating those tags with low confidence, achieving a better performance by mitigating the hallucination problem. Furthermore, this confidence score can be directly set as a relevance metric for the downstream tasks.

Tag Confidence Training. In order to make the confidence score more consistent with the requirements of information retrieval, we construct a confidence training dataset $\mathcal{D}'$ as:

$$\begin{aligned}\mathcal{D}' &= \{(x_i', y_i')\}_{i=1}^M, \\ x_i' &= \text{Prompt}_c(c_i, t_i) \,, \\ y_i' &\in \{\text{“}Yes\text{”}, \text{“}No\text{”}\} \,,\end{aligned} \tag{11}$$

where y_i is annotated by experts and M is the size of training dataset. Then we leverage the causal language modeling objective, which is the same as Equation (8) to perform supervised fine-tuning. In that case, the confidence score predicted by this module aligns with the requirements of the information retrieval systems, thereby facilitating the calibration of incorrect tags.

4 EXPERIMENTS

In this section, we conduct extensive experiments to answer the following research questions:

- **RQ1** How does LLM4Tag perform in comparison to existing tagging algorithms?
- **RQ2** How effective is the graph-based tag recall module?
- **RQ3** Does the injection of domain-specific knowledge enhance the tagging performance?
- **RQ4** What is the impact of the tag confidence calibration module?

4.1 Experimental Settings

4.1.1 Dataset. We conducted experiments on a mainstream information distribution platform with hundreds of millions of users and sampled three representative industrial datasets from online logs to ensure consistency in data distribution, containing two types of tasks: (1) multi-tag task (*Browser News*), and (2) single-tag task (*Advertisement Creatives* and *Search Query*).

- **Browser News** dataset includes popular news articles and user-generated videos, primarily in the form of text, images, and short videos. This is a multi-tag task, wherein the objective is to select multiple appropriate tags for each content from a massive tag repository (more than 100,000 tags). Around 30,000 contents are randomly collected as the testing dataset through expert annotations.
- **Advertisement Creatives** dataset includes ad creatives, including cover images, copywriting, and product descriptions from advertisers. The task for this dataset is a single-tag task, where we need to select the most relevant tag to the advertisement from a well-designed tag repository (more than 1,000 tags) and we collect around 10,000 advertisements randomly as the testing dataset through expert annotation.
- **Search Query** dataset primarily consists of user search queries from a web search engine, used for user intent classification. The task for this dataset is also a single-tag task, where the most probable intent needs to be selected as the tag for each query. The size of the tag repository is about 1,000, and 2,000 queries are collected and manually tagged as the testing dataset.

Table 1: Performance comparison of different methods. Note that different tasks, multi-tag tasks (Brower News) and single-tag tasks (Advertisement Creatives and Search Query), have different metrics. The best result is given in bold, and the second-best value is underlined. "RI" indicates the relative improvements of LLM4Tag over the corresponding baseline.

Model	Browser News				Advertisement Creatives				Search Query			
	Acc@1	Acc@2	Acc@3	RI	Precision	Recall	F1	RI	Precision	Recall	F1	RI
BGE	0.7427	0.6584	0.5976	29.8%	0.7817	0.7396	0.7601	18.5%	0.6364	0.5122	0.5676	56.2%
GTE	0.7292	0.6507	0.5941	31.3%	0.7369	0.7026	0.7194	25.0%	0.6129	0.4634	0.5278	67.9%
CONAN	0.7568	0.6814	0.6266	25.5%	0.7491	0.7194	0.7339	22.6%	0.6056	0.5244	0.5621	57.9%
TagGPT	0.8351	0.7813	0.7424	9.5%	0.8454	0.7997	0.8219	9.4%	0.8421	0.7805	0.8101	9.7%
ICXML	0.8398	0.7883	0.7560	8.4%	0.8492	0.8025	0.8252	9.0%	0.8600	0.7840	0.8202	8.3%
LLM4TC	<u>0.8602</u>	<u>0.8069</u>	<u>0.8235</u>	3.7%	<u>0.8726</u>	<u>0.8245</u>	<u>0.8479</u>	6.1%	<u>0.9028</u>	<u>0.8025</u>	<u>0.8497</u>	4.5%
LLM4Tag	**0.9041**	**0.8511**	**0.8273**	-	**0.9138**	**0.8857**	**0.8995**	-	**0.9325**	**0.8485**	**0.8885**	-

4.1.2 Baselines. To evaluate the superiority and effectiveness of our proposed model, we compare LLM4Tag with two classes of existing models:

- **Traditional Methods** that encode the contents and tags by leveraging pre-trained language models and select the most relevant tags according to cosine distance for each content as the result. Here we compare three different pre-trained language models. **BGE** [30] pre-trains the models with retromae on large-scale pairs data using contrastive learning. **GTE** [19] further proposes multi-stage contrastive learning to train the text embedding. **CONAN** [16] maximizes the utilization of more and higher-quality negative examples to pre-train the model.
- **LLM-Enhanced Methods** that utilize large language models to assist the tag generation. **TagGPT** [15] proposes a zero-shot automated tag extraction system through prompt engineering via LLMs. **ICXML** [35] introduces a two-stage tag generation framework, involving generation-based label shortlisting and label reranking through in-context learning. **LLM4TC** [5] further leverages fine-tuning using domain knowledge to improve the performance of tag generation.

4.1.3 Evaluation Metrics. For multi-tag tasks, due to the excessive number of tags (millions), we can not annotate all the correct tags and thus only directly judge whether the results generated by the model are correct or not. In this case, we define Acc@k to evaluate the performance:

$$\begin{aligned} \text{Acc@k} &= \frac{1}{N'}\sum_{i=1}^{N'}\sum_{j=1}^{k'}\frac{\mathbb{I}(T_i[j])}{k'}, \\ k' &= \min(k, len(T_i)), \\ \mathbb{I}(T_i[j]) &= \begin{cases} 1, & T_i[j] \text{ is right}, \\ 0, & \text{otherwise}, \end{cases} \end{aligned} \tag{12}$$

where $T_i[j]$ is the j-th generated tag of the i-th content and N' is the size of test dataset. It is worth noticing there exists contents that do not have k proper tags, thus we allow the number of generated tags to be less than k.

For the single-tag task, we adopt Precision, Recall, and F1 following previous works [5, 15]. Higher values of these metrics indicate better performance.

Moreover, we report Relative Improvement (RI) to represent the relative improvement our model achieves over the compared models. Here we calculate the average RI of the above all metrics.

4.1.4 Implementation Details. In the selection of LLMs, we select Huawei's large languge model PanGu-7B [28, 31]. For the graph-based tag recall module, we choose BGE [30] as the encoder model. δ_{c-t} and δ_{c-c} are set as 0.5 and 0.8, respectively. Besides, we set maximum recall numbers for different meta-paths, 15 for C2T meta-path and 5 for C2C2T meta-path. For knowledge-enhanced tag generation module, the size of the training dataset in long-term supervised knowledge injection contains approximately $10,000$ annotated samples and the tuning is performed every two weeks. As for the short-term retrieved knowledge injection, the retrievable database is updated in real-time and we retrieve at most 3 relevant samples/segments for in-context learning injection and augmented generation injection, respectively. For the tag confidence calibration module, we eliminate tags with confidence scores less than 0.5 and rank the remaining tags in order of confidence scores as the result.

4.2 Result Comparison & Deployment (RQ1)

Table 1 summarizes the performance of different methods on three industrial datasets, from which we have the following observations:

- **Leveraging large language models (LLMs) brings benefit to model performance.** TagGPT, ICXML, and LLM4TC, utilize LLMs to assist the tag generation, achieving better performances than other small language models (SLMs), such as BGE, GTE, and CONAN. This phenomenon indicates that the world knowledge and reasoning capabilities of LLMs enable better content understanding and tag generation, significantly improving tagging effectiveness.
- **Introducing domain knowledge can significantly improve performance.** Although LLMs benefit from general world knowledge, there remains a significant gap compared with domain-specific knowledge. Therefore, LLM4TC injects domain knowledge by fine-tuning the LLMs and achieves better performance

than other baselines in all metrics, which validates the importance of domain knowledge injection.

- **The superior performance of LLM4Tag.** We can observe from Table 1 that LLM4Tag yields the best performance on all datasets consistently and significantly, validating the superior effectiveness of our proposed LLM4Tag. Concretely, LLM4Tag beats the best baseline by **3.7%**, **6.1%**, and **4.5%** on three datasets, respectively. This performance improvement is attributed to the advanced nature of our LLM4Tag, including more comprehensive graph-based tag recall, deeper domain-specific knowledge injection, and more reliable confidence calibration.
- **Notably, LLM4Tag has been deployed online and covers all the traffic.** We randomly resampled the online data, and the online report shows consistency between the improvements in the online metrics and those observed in the offline evaluation. Now, LLM4Tag has been deployed in the content tagging system of these three online applications, serving hundreds of millions of users daily.

4.3 The Effectiveness of Graph-based Tag Recall Module (RQ2)

In this subsection, we compare our proposed graph-based tag recall module with match-based recall to validate the effectiveness of candidate tags retrieval over the Browser News Dataset. For fairness, both methods use the same pre-trained language model BGE to encode contents and tags, and the number of candidate tags is fixed as 20. We define two metrics to evaluate the performance: *#Right* means the average number of correct tags in candidate tags, and *HR#k* means the proportion of cases where at least k correct tags are hit in the candidate tag set.

Table 2: Performance comparison between different recall types over the Browser News Dataset.

Recall Type	#Right	HR#1	HR#2	HR#3
Match-based	4.48	0.9586	0.8841	0.7643
Ours	5.37	0.9745	0.9212	0.8425

As shown in Table 2, we can find that our graph-based recall method can significantly improve the quality of candidate tags. The metrics #Right and HR#3 increase by 19.8% and 10.2% respectively, which demonstrates that our method yields a more complete and comprehensive candidate tag set via a meta-path-based multi-hop recall mechanism. Moreover, the lifting of HR#1 illustrates that our method can recall the correct tags when the match-based method encounters challenges in hard cases and fails to select the relevant tags.

Besides, to verify the effectiveness and interpretability of our proposed graph-based tag recall, we randomly select some cases in our deployed tagging scenario and visualize the recall results in Figure 6. It can be observed that, when match-based recall fails to select the correct tags for some challenging cases, our method effectively retrieves accurate tags by C2C2T meta-path multi-hop traversal in the graph, thus avoiding missing correct tags due to the limited capabilities of SLMs.



Figure 6: The online cases for verifying the effectiveness of graph-based tag recall.

4.4 The Effectiveness of Knowledge-enhanced Tag Generation (RQ3)

In order to systematically evaluate the contribution of domain knowledge-enhanced tag generation (KETG) in our framework, we have designed the following variants:

- **LLM4Tag (w/o KETG)** removes both long-term supervised knowledge injection (LSKI) and short-term retrieved knowledge injection (SRKI), and selects tags using native LLMs.
- **LLM4Tag (w/o LSKI)** removes LSKI and only maintains SRKI to inject the short-term domain-specific knowledge.
- **LLM4Tag (w/o SRKI)** removes SRKI and only maintains LSKI to inject the long-term domain-specific knowledge.
- **LLM4Tag (Ours)** incorporates both LSKI and SRKI to inject the long/short-term domain-specific knowledge.

Figure 7 presents the comparative results on Browser News Dataset, revealing three key findings:

- The complete framework achieves optimal performance, demonstrating the synergistic value of combining supervised fine-tuning in long-term supervised knowledge injection with non-parametric short-term retrieved knowledge injection.
- The removal of either component in knowledge-enhanced tag generation module causes measurable degradation. Among them, the removal of long-term knowledge results in a greater decline, indicating that long-term knowledge may cover a broader range of domain-specific knowledge and highlighting the importance of SFT in model knowledge injection.

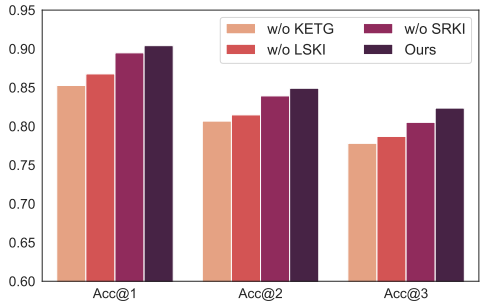


Figure 7: Ablation study about the effectiveness of knowledge-enhanced tag generation module in LLM4Tag.

- The most basic variant (w/o KETG) exhibits the lowest performance, highlighting the crucial role of domain adaptation in specialized tagging tasks within information retrieval systems.

4.5 The Effectiveness of Tag Confidence Calibration (RQ4)

To validate the effectiveness of the tag confidence calibration module, we evaluate model performance on the Browser News Dataset and use different confidence thresholds to achieve different pruning rates. Here we define a metric *Coverage@k* to evaluate the cover rate of final results as:

$$\begin{aligned} \text{Coverage@}k &= \frac{1}{N'}\sum_{i=1}^{N'}\mathbb{I}\left(|T_i| \geq k\right), \\ \mathbb{I}\left(|T_i| \geq k\right) &= \begin{cases} 1, & |T_i| \geq k, \\ 0, & \text{otherwise}, \end{cases} \end{aligned} \tag{13}$$

where T_i is the result tags of the i-th content and N' is the size of testing dataset.

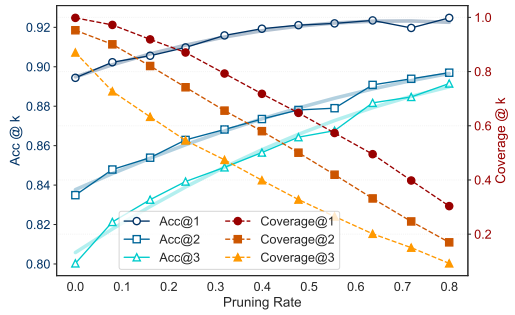
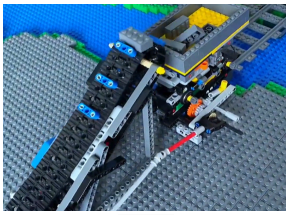


Figure 8: Model Accuracy vs. Tag Coverage for Different Pruning Rates.

As shown in Figure 8, our experimental results indicate that when we increase the pruning rate by setting a larger confidence threshold, the Acc@k is significantly boosted while the Coverage@k continues to decrease, which demonstrates the effectiveness of our proposed tag confidence calibration module. Additionally, as the pruning rate increases, the accuracy gains gradually slow down. This characteristic allows us to set an appropriate confidence threshold in practical deployment scenarios to achieve a balance between prediction accuracy and tag coverage.



Case A
Title: Update of lego city!

Tag	Confidence Score
Lego	0.93046
Freight Train	0.04380
Building Blocks	0.97737



Case B
Title: The winner of a bullfight.

Tag	Confidence Score
Buffalo	0.88720
Bullfight	0.84662
Bulldog	0.13784



Case C
Title: How much can a vampire hunter's kit cost?

Tag	Confidence Score
Antiques	0.92117
Repair Tools	0.31911
Religious Culture	0.03445



Case D
Title: A group of orca is hunting.

Tag	Confidence Score
Orca	0.90375
Hunting	0.89805
Seals	0.55549

Figure 9: The online cases of tag confidence calibration module. Tags with low confidence are highlighted in red.

Furthermore, we randomly select some cases in our deployed tagging scenario and visualize them with confidence scores in Figure 9. We find that in Cases A, B, and C, irrelevant tags such as "Freight Train," "Bulldog," and "Religious Culture" receive low confidence scores and will be calibrated by our model, and in Case D, the weak-relevant tag, "Seals", which is a non-primary entity in the figure, receive a medium confidence score and will be ranked low in the final results, which further demonstrates the superiority of tag confidence calibration module.

5 CONCLUSION

In this work, we propose an automatic tagging system based on Large Language Models (LLMs) named LLM4Tag with three key modules, characterized by completeness, continuous knowledge evolution, and quantifiability. Firstly, the graph-based tag recall module is designed to construct a small-scale relevant, comprehensive candidate tag set from a massive tag repository. Next, the knowledge-enhanced tag generation module is proposed to generate accurate tags with knowledge injection. Finally, the tag confidence calibration module is employed to generate reliable confidence tag scores. The significant improvements in offline evaluations have demonstrated its superiority and LLM4Tag has been deployed online for content tagging.

REFERENCES

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[2] Sajad Ahmadian, Milad Ahmadian, and Mahdi Jalili. 2022. A deep learning based trust-and tag-aware recommender system. *Neurocomputing* 488 (2022), 557–571.

[3] Kerstin Bischoff, Claudiu S Firan, Wolfgang Nejdl, and Raluca Paiu. 2008. Can all tags be used for search?. In *Proceedings of the 17th ACM conference on Information and knowledge management*. 193–202.

[4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[5] Youngjin Chae and Thomas Davidson. 2023. Large language models for text classification: From zero-shot learning to fine-tuning. *Open Science Foundation* (2023).

[6] Keunwoo Choi, George Fazekas, and Mark Sandler. 2016. Automatic tagging using deep convolutional neural networks. *arXiv preprint arXiv:1606.00298* (2016).

[7] Antonina Dattolo, Felice Ferrara, and Carlo Tasso. 2010. The role of tags for recommendation: a survey. In *3rd International Conference on Human System Interaction*. IEEE, 548–555.

[8] Ernesto Diaz-Aviles, Mihai Georgescu, Avaré Stewart, and Wolfgang Nejdl. 2010. Lda for on-the-fly auto tagging. In *Proceedings of the fourth ACM conference on Recommender systems*. 309–312.

[9] Ashraf Elnagar, Omar Einea, and Ridhwan Al-Debsi. 2019. Automatic text tagging of Arabic news articles using ensemble deep learning models. In *Proceedings of the 3rd international conference on natural language and speech processing*. 59–66.

[10] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).

[11] Manish Gupta, Rui Li, Zhijun Yin, and Jiawei Han. 2010. Survey on social tagging techniques. *ACM Sigkdd Explorations Newsletter* 12, 1 (2010), 58–72.

[12] Tokutaka Hasegawa and Shun Shiramatsu. 2021. BERT-Based Tagging Method for Social Issues in Web Articles. In *Proceedings of Sixth International Congress on Information and Communication Technology: ICICT 2021, London, Volume 1*. Springer, 897–909.

[13] Yuheng Huang, Jiayang Song, Zhijie Wang, Shengming Zhao, Huaming Chen, Felix Juefei-Xu, and Lei Ma. 2023. Look before you leap: An exploratory study of uncertainty measurement for large language models. *arXiv preprint arXiv:2307.10236* (2023).

[14] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *Comput. Surveys* 55, 12 (2023), 1–38.

[15] Chen Li, Yixiao Ge, Jiayong Mao, Dian Li, and Ying Shan. 2023. Taggpt: Large language models are zero-shot multimodal taggers. *arXiv preprint arXiv:2304.03022* (2023).

[16] Shiyu Li, Yang Tang, Shizhe Chen, and Xi Chen. 2024. Conan-embedding: General Text Embedding with More and Better Negative Samples. arXiv:2408.15710 [cs.CL] https://arxiv.org/abs/2408.15710

[17] Xin Li, Lei Guo, and Yihong Eric Zhao. 2008. Tag-based social interest discovery. In *Proceedings of the 17th international conference on World Wide Web*. 675–684.

[18] Yangning Li, Shirong Ma, Xiaobin Wang, Shen Huang, Chengyue Jiang, Hai-Tao Zheng, Pengjun Xie, Fei Huang, and Yong Jiang. 2024. Ecomgpt: Instruction-tuning large language models with chain-of-task tasks for e-commerce. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 18582–18590.

[19] Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. 2023. Towards general text embeddings with multi-stage contrastive learning. *arXiv preprint arXiv:2308.03281* (2023).

[20] Jianghao Lin, Xinyi Dai, Yunjia Xi, Weiwen Liu, Bo Chen, Hao Zhang, Yong Liu, Chuhan Wu, Xiangyang Li, Chenxu Zhu, et al. 2023. How can recommender systems benefit from large language models: A survey. *ACM Transactions on Information Systems* (2023).

[21] Pengfei Liu, Xipeng Qiu, and Xuanjing Huang. 2016. Recurrent neural network for text classification with multi-task learning. *arXiv preprint arXiv:1605.05101* (2016).

[22] Gilad Mishne. 2006. Autotag: a collaborative approach to automated tag assignment for weblog posts. In *Proceedings of the 15th international conference on World Wide Web*. 953–954.

[23] Şükrü Ozan and D Emre Taşar. 2021. Auto-tagging of short conversational sentences using natural language processing methods. In *2021 29th Signal Processing and Communications Applications Conference (SIU)*. IEEE, 1–4.

[24] Shahzad Qaiser and Ramsha Ali. 2018. Text mining: use of TF-IDF to examine the relevance of words to documents. *International Journal of Computer Applications* 181, 1 (2018), 25–29.

[25] Xiaofei Sun, Xiaoya Li, Jiwei Li, Fei Wu, Shangwei Guo, Tianwei Zhang, and Guoyin Wang. 2023. Text classification via large language models. *arXiv preprint arXiv:2305.08377* (2023).

[26] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).

[27] Peilu Wang, Yao Qian, Frank K Soong, Lei He, and Hai Zhao. 2015. A unified tagging solution: Bidirectional lstm recurrent neural network with word embedding. *arXiv preprint arXiv:1511.00215* (2015).

[28] Yunhe Wang, Hanting Chen, Yehui Tang, Tianyu Guo, Kai Han, Ying Nie, Xutao Wang, Hailin Hu, Zheyuan Bai, Yun Wang, et al. 2023. PanGu-π: Enhancing Language Model Architectures via Nonlinearity Compensation. *arXiv preprint arXiv:2312.17276* (2023).

[29] Zhiqiang Wang, Yiran Pang, and Yanbin Lin. 2023. Large language models are zero-shot text classifiers. *arXiv preprint arXiv:2312.01044* (2023).

[30] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-Pack: Packaged Resources To Advance General Chinese Embedding. arXiv:2309.07597 [cs.CL]

[31] Wei Zeng, Xiaozhe Ren, Teng Su, Hui Wang, Yi Liao, Zhiwei Wang, Xin Jiang, ZhenZhang Yang, Kaisheng Wang, Xiaoda Zhang, et al. 2021. Pangu-α: Large-scale autoregressive pretrained Chinese language models with auto-parallel computation. *arXiv preprint arXiv:2104.12369* (2021).

[32] Ye Zhang and Byron Wallace. 2015. A sensitivity analysis of (and practitioners' guide to) convolutional neural networks for sentence classification. *arXiv preprint arXiv:1510.03820* (2015).

[33] Zi-Ke Zhang, Tao Zhou, and Yi-Cheng Zhang. 2011. Tag-aware recommender systems: a state-of-the-art survey. *Journal of computer science and technology* 26, 5 (2011), 767–777.

[34] Yutao Zhu, Huaying Yuan, Shuting Wang, Jiongnan Liu, Wenhan Liu, Chenlong Deng, Haonan Chen, Zheng Liu, Zhicheng Dou, and Ji-Rong Wen. 2023. Large language models for information retrieval: A survey. *arXiv preprint arXiv:2308.07107* (2023).

[35] Yaxin Zhu and Hamed Zamani. 2023. ICXML: An in-context learning framework for zero-shot extreme multi-label classification. *arXiv preprint arXiv:2311.09649* (2023).